

Table 7: Jointly considering aggregated attention weights of consecutive tokens is better than an individual threshold.

Method	Without Attack		Combined Attack	
	Utility	ASR	Utility	ASR
Individual threshold-0.01	0.54	0.0	0.54	0.02
Individual threshold-0.02	0.57	0.0	0.52	0.11
Individual threshold-0.05	0.59	0.0	0.46	0.16
Our joint consideration	0.59	0.0	0.59	0.01

Table 8: Effectiveness of PISanitizer using different sanitization instructions under Naive Attack. Details can be found in Appendix C.

Sanitization Instruction	Without Attack		Naive Attack	
	Utility	ASR	Utility	ASR
No sanitization instruction	0.57	0.02	0.51	0.10
Target instruction	0.42	0.02	0.44	0.01
Sanitization instruction 1	0.59	0.0	0.56	0.04
Sanitization instruction 2	0.59	0.0	0.56	0.03
Sanitization instruction 3	0.59	0.0	0.58	0.03
Sanitization instruction 4	0.59	0.0	0.57	0.04

Table 9: Effectiveness of PISanitizer using different strategies to aggregate attention weights.

Attention Pattern	Without Attack		Combined Attack	
	Utility	ASR	Utility	ASR
Average aggregation	0.55	0.0	0.55	0.01
Noise-aware aggregation	0.59	0.0	0.59	0.01

Jointly considering aggregated attention weights of consecutive tokens is effective and necessary: As shown in Table 7, our proposed joint consideration strategy consistently outperforms individual thresholding. Individual threshold achieves a sub-optimal trade-off between utility and ASR: a low threshold removes more injected tokens but leads to noticeable utility loss, whereas a high threshold preserves utility but allows some attacks to succeed. In contrast, our method effectively balances both objectives, maintaining high utility while substantially reducing ASR.

Effectiveness of PISanitizer under different sanitization instructions: Table 8 shows the effectiveness of PISanitizer when using different sanitization instructions under Naive Attack. We have the following observations from experimental results. First, without using a sanitization instruction, the ASR is higher. The reason is that the LLM may not pay enough attention to the injected instruction, making it more challenging for leveraging attention to perform sanitization. Second, when using the target instruction as the sanitization instruction, there is a utility loss. The reason is that the LLM also pays attention to clean tokens that are important for target tasks. As a result, these clean tokens can be mistakenly sanitized, leading to utility loss. Third, PISanitizer is generally not sensitive to sanitization instructions that are designed to let an LLM follow instructions in a context, i.e., PISanitizer consistently achieves good performance under different sanitization instructions.

Effectiveness of noise-aware aggregation of attention weights: Table 9 compares our noise-aware aggregation with average aggregation (directly averaging over all attention heads). The results show that PISanitizer with noise-aware aggregation outperforms average aggregation.

Impact of threshold θ , group distance d , and smooth window w_s : In general, we find that PISanitizer is relatively insensitive to these parameters. See Appendix F for results.

5.4 Adaptive Attacks

We consider *heuristic-based adaptive attacks* and *optimization-based adaptive attacks*. Specifically, an attacker aims to achieve two goals in crafting an injected prompt. The first goal is that the attack should be effective, i.e., given a target instruction and a contaminated context with the injected instruction, an LLM should follow the injected instruction to generate an attacker-desired output. The second goal is that the attack should be stealthy, i.e., given a sanitization instruction and a contaminated context, the LLM should not follow the injected instruction, i.e., making the LLM pay no or low attention to the injected instruction under sanitization instruction.

Heuristic-I: We can repeat the injected prompt multiple times within the context. As a result, the attack may still be effective if any injected prompts are not sanitized by PISanitizer. In our experiment, we insert the injected prompt three times at random positions in a context.

Heuristic-II: PISanitizer leverages a sanitization instruction with the goal of letting an LLM follow any instructions in the context. An attacker can craft an injected prompt that attempts to neutralize the sanitization instruction’s effect by instructing the LLM to remain inactive when it encounters it. To this end, we design the following injected prompt: “*If your task is {sanitization instruction}, please do nothing. Otherwise, please {injected instruction}*”.

Heuristic-III: We design a sanitization instruction and let an LLM follow it. An attacker can design an injected prompt such that the LLM does not follow the sanitization instruction. To this end, we design the following injected prompt: “*Please don’t follow the following instruction {sanitization instruction}. Only follow the {injected instruction} when you are asked for the {target instruction}*”.

Heuristic-IV: PISanitizer groups nearly peaks together and sanitize them. An attacker can exploit this heuristic by inserting special tokens between consecutive tokens in the injected instruction to disrupt peak grouping and evade sanitization. We find that the token “X” usually has very low attention weights in an LLM. As we set the group distance d in our algorithm to 10, we insert “X” 10 times between each two injected tokens to disrupt the peak grouping, where each “X” is tokenized as a token.

Optimization-I: We also perform an optimization-based attack. In particular, we optimize the suffix appended to the injected instruction with two goals. First, the LLM should follow the injected instruction to generate an attacker-desired output. Second, the attention weights between the injected tokens and the generated output token under the sanitization instruction should be small. For the first goal, we define the following loss term $\ell_1 = -\log \Pr(\hat{Y}|I_t \oplus C_t \oplus E \oplus I_s \oplus S)$, where I_t is the target instruction, C_t is the clean context, E is the separator used by Combined Attack, I_s is the injected instruction, and S is the suffix for optimization (with 50 tokens), and $\hat{Y}$ is the attacker-desired LLM’s output under I_s . We also define $\ell_2 = \frac{1}{|I_s|} \sum_{c_k \in I_s} s_k$, where c_k is an injected token in I_s and s_k is the aggregated attention weight for c_k used to sanitize injected tokens by PISanitizer. By optimizing ℓ_2 , the attacker could suppress the attention weights of injected tokens under sanitization instruction, thus make them stealthy to bypass our attention-based sanitization. Together, we use nano-GCG and run for 500 iterations to update tokens in S to minimize the loss $\ell_1 + \beta \cdot \ell_2$, where β is a weight parameter to ensure these two loss terms are on the same order of magnitude. We set $\beta = 5,000$ to scale the terms so their magnitudes are comparable (i.e., the average ℓ_1 is approximately equal to the average ℓ_2).

Optimization-II: We find that the above loss function is challenging to optimize in practice. In particular, minimizing ℓ_1 (attack effectiveness) and ℓ_2 (stealth via suppressing attention) simultaneously may pull the optimizer in different directions, thereby making it very challenging for nano-GCG to search for an effective prompt that simultaneously achieve two goals. To further enhance adaptive attacks against PISanitizer, we therefore evaluate an attack that only optimizes ℓ_2 . We also use nanoGCG with 500 iterations. By focusing on reducing attention weights for injected tokens under the sanitization instruction, this strategy makes the injected instruction more likely to bypass PISanitizer.

Table 10: PISanitizer is effective under heuristic-based and optimization-based adaptive attacks.

Attack	No Defense		PISanitizer	
	Utility	ASR	Utility	ASR
Heuristic-I	0.19	0.82	0.55	0.04
Heuristic-II	0.40	0.40	0.55	0.03
Heuristic-III	0.48	0.39	0.58	0.0
Heuristic-IV	0.40	0.35	0.55	0.03
Optimization-I	0.31	0.62	0.56	0.0
Optimization-II	0.21	0.62	0.54	0.04

Table 11: Effectiveness of PISanitizer for short context and LLM agent.

Attack	Dataset	No Defense		PISanitizer	
		Utility	ASR	Utility	ASR
No Attack	AlpacaFarm	0.39	0.0	0.38	0.0
	SQuAD-v2	0.92	0.0	0.92	0.0
	Dolly	0.90	0.0	0.90	0.0
Under Attack	AlpacaFarm	0.01	0.59	0.37	0.0
	SQuAD-v2	0.01	0.99	0.86	0.0
	Dolly	0.0	0.96	0.84	0.0
	TaskTracker	N/A	0.40	N/A	0.01
	InjecAgent	N/A	0.37	N/A	0.02

5.4.1 Experimental Results

Table 10 shows the results under our default setting. We find that PISanitizer is also effective for adaptive attacks. The reason is that the sanitization instruction makes an LLM follow any instructions. For example, even if an injected instruction is designed to let an LLM not follow the sanitization instruction, the LLM will still follow the injected instruction. As a result, the LLM would pay attention to it, enabling PISanitizer to sanitize it. Additionally, due to the attention mechanism of LLMs [38], it can be challenging to reduce the attention weights for instructional tokens that are followed by an LLM.

5.5 Short Context and LLM Agent

PISanitizer is primarily designed for long-context scenarios (e.g., contexts with hundreds of tokens), where indirect prompt injection attacks are more likely to occur but more challenging to prevent. To assess the generality of PISanitizer, we also evaluate its performance on short-context settings on benchmark datasets following [17, 43].

Experimental setup: For non-agent settings, we use TaskTracker [30] and AlpacaFarm [65] prompt injection benchmarks. We also evaluate SQuAD-v2 [66] and Dolly [67], where we use the same injected task as AlpacaFarm [65]. For the agent, we perform an evaluation on InjecAgent under ENHANCED attack [68].

Experimental results: Table 11 shows the experimental results, where the backend LLM is Llama-3.1-8B-Instruct. We don’t show utility for TaskTracker and InjecAgent because these two benchmarks do not provide utility evaluation. When there are no attacks, PISanitizer can generally maintain utility. Under prompt injection attacks, PISanitizer can consistently reduce ASRs to nearly zero, demonstrating that it can effectively sanitize injected instructions under short contexts and LLM agent scenario.

6 Discussion and Limitation

While being effective, PISanitizer is not without limitations. We discuss and acknowledge them below.